

Unfish from Scratch: A Gentle Introduction to Programming

The Complete Beginner's Guide & Hands-on Tutorial

The Unfish Project & Educational Working Group

October 2026 - Version 2.1.0

Contents

1	Introduction: Welcome to Unfish	2
1.0.1	Why Unfish?	2
2	Chapter 1: Your First Unfish Program	2
2.0.1	The <code>say</code> Statement	2
2.0.2	Printing Without a Newline: <code>print</code>	2
3	Chapter 2: Variables, Arithmetic and Types	3
3.0.1	Creating Variables with <code>let</code>	3
3.0.2	Basic Data Types	3
3.0.3	Inspecting Types with <code>type_of</code>	3
3.0.4	Arithmetic Operations	3
3.0.5	Formatted String Interpolation (<code>f"..."</code>)	3
4	Chapter 3: Controlling the Flow	4
4.0.1	Conditional Branching: <code>if</code> , <code>elif</code> , <code>else</code>	4
4.0.2	Comparison and Logical Operators	4
5	Chapter 4: Repetition & Loops	4
5.0.1	The Educational <code>repeat</code> Loop	4
5.0.2	The <code>while</code> Loop	4
5.0.3	The <code>for ... in</code> Loop	5
6	Chapter 5: Functions & Lambdas	5
6.0.1	Declaring Functions	5
6.0.2	Default Parameters	5
6.0.3	Anonymous Functions and <code>fn</code> Lambdas	5
7	Chapter 6: Collections: Arrays & Maps	6
7.0.1	Arrays (Lists)	6
7.0.2	Powerful Array Combinators: <code>map</code> , <code>filter</code> , <code>reduce</code>	6
7.0.3	Hash Maps (Dictionaries)	6
8	Chapter 7: Structs & Object-Oriented Programming	7

9 Chapter 8: Resilient Error Handling	8
10 Chapter 9: Using Standard Library Modules	8
10.0.1 The <code>sys</code> Module	8
10.0.2 The <code>fs</code> (File System) Module	8
10.0.3 The <code>time</code> Module	9
11 Chapter 10: Hands-On Practice Exercises	9
11.0.1 Challenge 1: Marine Creature Counter	9
11.0.2 Challenge 2: Coral Growth Tracker	9
12 Summary & What's Next	10

1 Introduction: Welcome to Unfish

Welcome to **Unfish**! Whether this is your first time writing code or you are transitioning from visual blocks, Unfish is designed to make learning programming joyful, clear, and powerful.

1.0.1 Why Unfish?

Many programming languages are filled with confusing punctuation like semicolons (;), curly braces ({}), and confusing setup rituals. Unfish gets rid of all that clutter. It uses **clean indentation** to organize code, just like a well-formatted outline in an essay.

Best of all, what you learn in this guide doesn't stop at simple scripts. The exact same Unfish code you write here can run on virtual machines, compile to standalone C code, and even power microcontrollers in robots!

2 Chapter 1: Your First Unfish Program

2.0.1 The `say` Statement

In Unfish, sending text to your screen is as simple as using the `say` statement:

```
say "Hello, Ocean!"
```

To run this, save it in a file called `hello.unfish` and run it from your terminal:

```
unfish run hello.unfish
```

Output:

```
Hello, Ocean!
```

2.0.2 Printing Without a Newline: `print`

While `say` automatically adds a line break after its output, `print` keeps the cursor on the same line:

```
print("Loading: ")
print("100%\n")
```

3 Chapter 2: Variables, Arithmetic and Types

3.0.1 Creating Variables with `let`

A variable is a labeled container that stores information. In Unfish, you declare variables using the `let` keyword:

```
let player_name = "Coral"
let score = 100
let speed = 2.5
let is_active = true
```

3.0.2 Basic Data Types

Unfish has six core fundamental data types: 1. **Numbers**: Both whole integers (42) and decimal numbers (3.14159) are numbers. 2. **Strings**: Sequences of characters enclosed in double quotes ("Deep Blue"). 3. **Booleans**: Logical truth values (`true` or `false`). 4. **Null**: Represents the absence of a value (`null`). 5. **Arrays**: Ordered lists of values ([10, 20, 30]). 6. **Maps**: Key-value dictionaries ({`"species": "Clownfish", "speed": 1.5`}).

3.0.3 Inspecting Types with `type_of`

You can ask Unfish for the type of any value using `type_of()`:

```
say type_of(42)      # "number"
say type_of("Coral") # "string"
say type_of(true)    # "boolean"
say type_of(null)    # "null"
```

3.0.4 Arithmetic Operations

Unfish supports all standard mathematical operations:

```
let a = 10
let b = 3

say a + b # Addition: 13
say a - b # Subtraction: 7
say a * b # Multiplication: 30
say a / b # Division: 3.3333333333333335
say a % b # Modulo (remainder): 1
```

3.0.5 Formatted String Interpolation (`f"..."`)

Embedding values directly inside strings is easy with formatted f-strings:

```
let fish_name = "Nemo"
let depth = 45.2
say f"The fish {fish_name} is swimming at depth {depth} meters."
```

4 Chapter 3: Controlling the Flow

Programming becomes interesting when our code can make decisions.

4.0.1 Conditional Branching: `if`, `elif`, `else`

Unfish uses indentation (4 spaces) to mark which lines belong inside a block:

```
let oxygen_level = 75

if oxygen_level > 80:
    say "Oxygen level is optimal."
elif oxygen_level > 50:
    say "Oxygen level is moderate."
else:
    say "Warning: Low oxygen level!"
```

4.0.2 Comparison and Logical Operators

- Comparison: `==` (equal), `!=` (not equal), `<` (less), `<=` (less or equal), `>` (greater), `>=` (greater or equal).
- Logic: `and`, `or`, `not`.

```
let temperature = 24
let salinity = 35

if temperature >= 20 and salinity <= 40:
    say "Lagoon conditions are safe for marine life."
```

5 Chapter 4: Repetition & Loops

When you need to repeat actions, Unfish gives you three easy tools.

5.0.1 The Educational `repeat` Loop

When you know exactly how many times an action should happen:

```
repeat 3:
    say "Bubbles rising..."
```

5.0.2 The `while` Loop

Repeats as long as a condition remains true:

```
let countdown = 3
while countdown > 0:
```

```
say f"Launching in {countdown}..."
countdown -= 1
say "Liftoff!"
```

5.0.3 The for ... in Loop

Iterates through every element of a list:

```
let reef_creatures = ["Starfish", "Turtle", "Anemone"]

for creature in reef_creatures:
  say f"Spotted a {creature} on the reef!"
```

You can also loop over a range of numbers using `range(start, end)`:

```
for i in range(1, 4):
  say f"Wave #{i}"
```

6 Chapter 5: Functions & Lambdas

Functions let you write a piece of logic once and reuse it anywhere.

6.0.1 Declaring Functions

You can declare a function using either `function` or the concise `fn` keyword:

```
fn calculate_distance(x, y):
  return sqrt(x * x + y * y)

let dist = calculate_distance(3, 4)
say f"Distance: {dist}" # 5
```

6.0.2 Default Parameters

Parameters can have default values if omitted by the caller:

```
fn greet(name = "Explorer"):
  say f"Welcome aboard, {name}!"

greet("Alice") # "Welcome aboard, Alice!"
greet()        # "Welcome aboard, Explorer!"
```

6.0.3 Anonymous Functions and fn Lambdas

You can pass quick, unnamed functions as arguments to other functions:

```
let numbers = [1, 2, 3, 4]
let doubled = map(numbers, fn(x): x * 2)
say doubled # [2, 4, 6, 8]
```

7 Chapter 6: Collections: Arrays & Maps

7.0.1 Arrays (Lists)

Arrays are ordered, dynamic collections:

```
let school = ["Clownfish", "Blue Tang"]

push(school, "Angelfish") # Add to the end
say len(school)           # 3
say school[0]             # "Clownfish"

let last = pop(school)    # Removes "Angelfish"
say last                  # "Angelfish"
```

7.0.2 Powerful Array Combinators: map, filter, reduce

```
let depths = [12, 45, 80, 22, 95]

# Keep only deep dives (> 30 meters)
let deep = filter(depths, fn(d): d > 30)
say deep # [45, 80, 95]

# Compute average depth
let total = reduce(deep, fn(acc, d): acc + d, 0)
say f"Average deep dive: {total / len(deep)}m"
```

7.0.3 Hash Maps (Dictionaries)

Maps store key-value pairs:

```
let reef_station = {
  "name": "Lagoon Alpha",
  "temperature": 26.5,
  "sensors_active": true
}

say reef_station["name"]      # "Lagoon Alpha"
say reef_station.temperature # 26.5 (dot notation also works!)

# Add or update keys
reef_station["pressure"] = 1.2
say keys(reef_station)       # ["name", "temperature", "sensors_active", "pressure"]
```

8 Chapter 7: Structs & Object-Oriented Programming

Structs let you create your own custom types with fields and methods. Let's build a real **School of Fish Simulation** step by step!

```
## Define the Fish blueprint
struct Fish:
    id
    x
    y
    speed

    # Method to move the fish based on a direction vector
    fn swim(self, dx, dy):
        return Fish(
            self.id,
            self.x + dx * self.speed,
            self.y + dy * self.speed,
            self.speed
        )

    # Method to calculate distance from the central coral reef (0, 0)
    fn distance_from_reef(self):
        return sqrt(pow(self.x, 2) + pow(self.y, 2))

# Create a school of 3 fish with different positions and speeds
let school = [
    Fish(1, 10, 20, 1.5),
    Fish(2, 14, 22, 1.2),
    Fish(3, 8, 19, 1.8)
]

say "Initial Fish Positions in the Lagoon:"
for f in school:
    say f"Fish #{f.id}: pos=({f.x}, {f.y}) - distance={round(f.distance_from_reef())}"

# A ocean current pushes all fish by (dx=5, dy=3)
let moved_school = map(school, fn(f): f.swim(5, 3))

say "\nPositions After Ocean Current:"
for f in moved_school:
    say f"Fish #{f.id}: now at ({f.x}, {f.y}) - new distance={round(f.distance_from_reef())}"
```

9 Chapter 8: Resilient Error Handling

Real-world programs occasionally run into unexpected problems (like a missing file or dividing by zero). Unfish provides structured `try`, `catch`, and `finally` blocks to handle errors gracefully:

```
fn safe_divide(numerator, denominator):
  try:
    if denominator == 0:
      raise "Cannot divide by zero!"
    return numerator / denominator
  catch err:
    say f"Error caught: {err}"
    return 0
  finally:
    say "Division operation finished."

say safe_divide(10, 2) # 5
say safe_divide(10, 0) # 0
```

The `finally` block is guaranteed to run even if an error occurs, making it perfect for closing files or cleaning up resources.

10 Chapter 9: Using Standard Library Modules

Unfish comes with built-in modules for interacting with the operating system, files, and time.

10.0.1 The `sys` Module

```
import sys

say f"Operating System: {sys.platform}"
say f"Unfish Version: {sys.version}"
say f"Current Directory: {sys.cwd()}"
```

10.0.2 The `fs` (File System) Module

```
import fs

# Writing to a file
fs.write_file("dive_log.txt", "Log #1: Reef inspection complete.\n")

# Reading from a file
let content = fs.read_file("dive_log.txt")
say f"Log contents:\n{content}"
```

10.0.3 The time Module

```
import time

let start = time.now()
time.sleep(50) # Pause for 50 milliseconds
let elapsed = time.diff_ms(start, time.now())
say f"Task completed in {elapsed}ms."
```

11 Chapter 10: Hands-On Practice Exercises

Test your skills with these fun challenges:

11.0.1 Challenge 1: Marine Creature Counter

Write a function `count_species(creatures, target)` that takes an array of creature names and counts how many times `target` appears.

Solution:

```
fn count_species(creatures, target):
  let total = 0
  for item in creatures:
    if item == target:
      total += 1
  return total

let bay = ["Turtle", "Dolphin", "Turtle", "Shark", "Turtle"]
say f"Turtles found: {count_species(bay, 'Turtle')}}" # 3
```

11.0.2 Challenge 2: Coral Growth Tracker

Write a struct `Coral` with fields `species` and `height`. Add a method `grow(years)` that increases height by `years * 1.5` and returns the new coral instance.

Solution:

```
struct Coral:
  species
  height

  fn grow(self, years):
    return Coral(self.species, self.height + years * 1.5)

let c = Coral("Brain Coral", 10.0)
let mature = c.grow(4)
say f"{mature.species} height after 4 years: {mature.height}cm" # 16.0cm
```

12 Summary & What's Next

Congratulations! You now have a solid command of the fundamentals of Unfish: * Clean indentation-based syntax * Variables, numbers, strings, and f-string interpolation * Loops, conditionals, and functions * Structs and object-oriented modeling * Error handling with `try-catch-finally` * Interacting with the real world using standard modules

To dive deeper into virtual machine design, compilers, and bare-metal systems programming, explore the **Unfish Technical Specification** and **The Unfish Book**!